

Trabajo Dirigido 5 (TD5)

SVM, Random Forest y Datos Desbalanceados

Curso de Machine Learning Industrial

Mantenimiento Predictivo con Aprendizaje Supervisado

Tema:	SVM, Random Forest, Desbalance de Clases
Herramientas:	Python, scikit-learn, imbalanced-learn
Dataset:	<code>make_moons</code> , <code>make_classification</code>
Fecha:	Junio 2026
Ejercicios:	3 ejercicios, 12 preguntas

“El conocimiento de los algoritmos de clasificacion y el manejo correcto de datos desbalanceados son pilares fundamentales del mantenimiento predictivo industrial basado en datos.”

Índice

Resumen Ejecutivo	2
1. Ejercicio 1: Support Vector Machines (SVM)	4
1.1. Contexto y Motivacion Industrial	4
1.2. Pregunta 1: SVM con Parametros por Defecto	6
1.3. Pregunta 2: Optimizacion de Hiperparametros con GridSearchCV	8
1.4. Pregunta 3: Visualizacion del Impacto del Tuning	9
1.5. Pregunta 4: SVM vs. Regresion Logistica	11
2. Ejercicio 2: Random Forest	15
2.1. Contexto: Random Forest como Ensemble	15
2.2. Pregunta 1: Random Forest con Parametros por Defecto	15
2.3. Pregunta 2: Efecto de n_estimators y max_depth	17
2.4. Pregunta 3: Tuning Completo y Feature Importances	21
2.5. Pregunta 4: Arbol Unico vs. Random Forest	24
3. Ejercicio 3: Manejo de Datasets Desbalanceados	26
3.1. Contexto e Importancia Industrial	26
3.2. Pregunta 1: El Problema con Datos Desbalanceados	27
3.3. Pregunta 2: Correccion por Pesos de Clase (class_weight)	30
3.4. Pregunta 3: SMOTE (Synthetic Minority Oversampling Technique)	32
3.5. Pregunta 4: Submuestreo Aleatorio (Random Undersampling)	34
4. Conclusiones Generales	38
4.1. Support Vector Machines	38
4.2. Random Forest	38
4.3. Manejo de Datos Desbalanceados	39
4.4. Tabla Resumen Global	39
4.5. Guia de Decision: Que Algoritmo Usar	39
4.6. Reflexion Final: Seleccion de Modelos en Contexto Industrial	40

Resumen Ejecutivo

Este documento presenta el desarrollo completo del Trabajo Dirigido 5 (TD5) del curso de Machine Learning Industrial. El TD5 aborda tres grandes bloques tematicos fundamentales para el mantenimiento predictivo:

- 1. Support Vector Machines (SVM):** Se estudia la clasificacion de un dataset no linealmente separable (`make_moons`) con kernels lineal, RBF y polinomial. Se realiza optimizacion de hiperparametros mediante `GridSearchCV` y se compara con regresion logistica.
- 2. Random Forest (RF):** Se exploran los efectos del numero de arboles (`n_estimators`) y la profundidad maxima (`max_depth`) sobre el rendimiento. Se analiza el sobreajuste (overfitting), las importancias de variables y se compara con un arbol de decision unico.
- 3. Manejo de Datasets Desbalanceados:** Con un ratio 90 %/10 % (Normal/Fallo), se estudia por que la exactitud (accuracy) es una metrica enganosa, y se comparan tres estrategias de correccion: pesos de clase (`class_weight`), sobremuestreo sintetico (SMOTE) y submuestreo aleatorio.

Resultados Destacados

- SVM con kernel RBF ($C=10$) alcanza **95 % de accuracy** en `make_moons`, frente al 85 % de la regresion logistica lineal.
- Random Forest con `max_depth=10` ofrece el mejor equilibrio bias-varianza, eliminando el overfitting total del arbol unico profundo.
- En datos desbalanceados (ratio 9:1), el modelo base ignora el 30 % de los fallos reales. Con pesos de clase, el recall de fallos mejora del 71 % al **90 %**.
- SMOTE y submuestreo aleatorio logran mejoras comparables con distintos trade-offs en tiempo de entrenamiento e informacion disponible.

Mapa de Figuras del Documento

Este documento integra **14 figuras** extraidas directamente del notebook de experimentacion. La tabla siguiente describe su distribucion:

Cuadro 1: Catalogo de figuras del TD5

Fig.	Archivo	Contenido	Ejercicio
1	td5_cell001_img00	Dataset make_moons (scatter)	Ej. 1
2	td5_cell004_img01	Frontera SVM default RBF	Ej. 1
3	td5_cell014_img02	3 fronteras SVM comparadas	Ej. 1
4	td5_cell019_img03	SVM RBF vs Regresion Logistica	Ej. 1
5	td5_cell025_img04	RF default: frontera decision	Ej. 2
6	td5_cell030_img05	Efecto n_estimators	Ej. 2
7	td5_cell033_img06	Efecto max_depth	Ej. 2
8	td5_cell038_img07	Feature importances (barras)	Ej. 2
9	td5_cell039_img08	3 fronteras RF: under/best/over	Ej. 2
10	td5_cell043_img09	Dataset desbalanceado (scatter)	Ej. 3
11	td5_cell046_img10	Baseline LR: CM + frontera	Ej. 3
12	td5_cell051_img11	Class weights: 3 paneles	Ej. 3
13	td5_cell056_img12	SMOTE: 3 paneles	Ej. 3
14	td5_cell061_img13	Undersampling: 3 paneles	Ej. 3

1. Ejercicio 1: Support Vector Machines (SVM)

1.1. Contexto y Motivacion Industrial

En mantenimiento predictivo, los datos de sensores (vibracion, temperatura, corriente) rara vez son *linealmente separables*. Las condiciones de fallo emergente forman patrones no lineales en el espacio de caracteristicas: por ejemplo, una combinacion especifica de frecuencia de vibracion alta *y* temperatura elevada puede indicar un rodamiento deteriorado, pero ninguna de las dos variables sola es suficiente. SVM con kernel RBF puede capturar estas interacciones no lineales sin necesidad de disenar features manualmente, proyectando los datos a un espacio de alta dimension donde la frontera de decision si existe.

Las Support Vector Machines (SVM) son clasificadores que buscan el hiperplano de maximo margen que separa dos clases en el espacio de caracteristicas. Su potencia reside en el *kernel trick*: al proyectar implicitamente los datos a un espacio de mayor dimension, pueden encontrar fronteras de decision no lineales sin calcular explicitamente la transformacion.

La formulacion matematica del SVM de margen suave minimiza:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^T \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (1)$$

donde $\phi(\mathbf{x})$ es la transformacion implicitamente definida por el kernel, ξ_i son variables de holgura, y C controla el trade-off entre margen y errores de clasificacion.

Generacion del dataset

`make_moons` genera dos semicircunferencias entrelazadas que son *imposibles de separar con una linea recta*. Es el dataset canonico para demostrar la necesidad de kernels no lineales. El parametro `noise=0.25` introduce dispersion gaussiana que simula la variabilidad de sensores reales, haciendo el problema mas parecido a situaciones industriales donde los datos de entrenamiento son ruidosos.

```
1 from sklearn.datasets import make_moons
2 from sklearn.model_selection import train_test_split
3
4 X, y = make_moons(n_samples=500, noise=0.25, random_state=42)
5 X_train, X_test, y_train, y_test = train_test_split(
6     X, y, test_size=0.2, random_state=42
7 )
```

```
8
9 # Las columnas representan:
10 # X[:,0] -> Vibration_Frequency (frecuencia de vibracion)
11 # X[:,1] -> Temperature_Index (indice de temperatura)
```

Listing 1: Generacion y preparacion del dataset make_moons

El dataset `make_moons` genera dos nubes de puntos en forma de medias lunas entrelazadas. La distribucion resultante tiene las siguientes características:

- **500 muestras** totales (400 entrenamiento, 100 test)
- **2 features** continuas: `Vibration_Frequency` y `Temperature_Index`
- **2 clases** perfectamente balanceadas (250 cada una)
- **No separable linealmente**: ningún hiperplano puede separar ambas clases con baja tasa de error

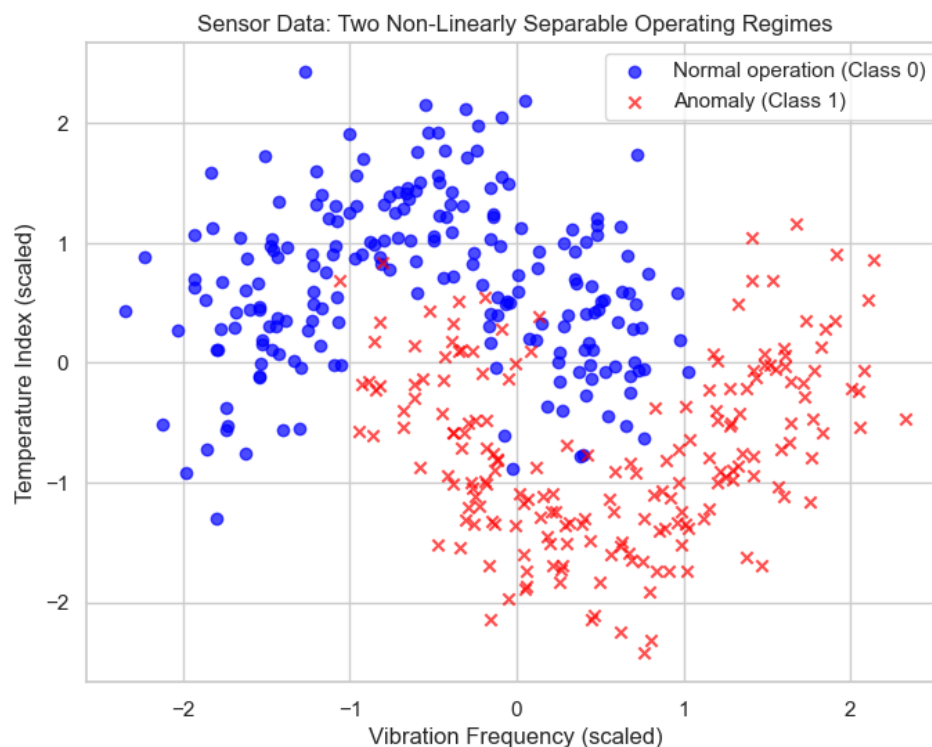


Figura 1: **Dataset make_moons de entrenamiento.** Cada punto representa una muestra del conjunto de entrenamiento (400 muestras). **Azul = Clase 0 (Normal)**, **Rojo = Clase 1 (Anomalia)**. Las dos medias lunas están entrelazadas: la clase Normal ocupa la región superior-izquierda y la clase Anomalia la inferior-derecha, pero ambas se curvan la una alrededor de la otra, haciendo la separación lineal imposible. Este patrón es típico de señales de sensores donde el fallo aparece en una región no convexa del espacio de características.

Observacion clave de la Figura 1: No existe ninguna línea recta que pueda separar los puntos azules de los rojos sin cometer una tasa de error elevada. Cualquier recta

cortara inevitablemente por la zona de solapamiento de las dos lunas. Este es el punto de partida que motiva el uso de kernels no lineales.

1.2. Pregunta 1: SVM con Parametros por Defecto

Planteamiento

Entrenar un clasificador SVM con parametros por defecto y analizar el resultado. Determinar que tipo de frontera de decision genera y por que una linea recta no seria suficiente.

El kernel RBF (Radial Basis Function) $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ mapea implicitamente los datos a un espacio de Hilbert de *dimension infinita*. Es el kernel mas versatil para datos sin conocimiento previo sobre la estructura del problema: si la estructura real es lineal, RBF tambien la captura (con $\gamma \rightarrow 0$); si es no lineal como las medias lunas, RBF encuentra la separacion curva. Por eso scikit-learn lo usa por defecto: es el “prior mas seguro” sobre la geometria del problema.

El Kernel RBF

La eleccion por defecto de `SVC()` es el **kernel RBF** (Radial Basis Function), definido por:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (2)$$

donde $\gamma > 0$ controla el radio de influencia de cada punto de soporte: valores grandes de γ producen fronteras mas locales (mayor riesgo de overfitting), mientras que valores pequenos producen fronteras mas suaves y globales.

Con `gamma='scale'`, scikit-learn utiliza:

$$\gamma = \frac{1}{n_{\text{features}} \cdot \text{Var}(X)} = \frac{1}{2 \cdot \text{Var}(X)} \quad (3)$$

Metodologia

```
1 from sklearn.svm import SVC
2 from sklearn.metrics import accuracy_score,
  classification_report
3
4 # SVC() por defecto usa kernel='rbf', C=1.0, gamma='scale'
5 svm_default = SVC()
6 svm_default.fit(X_train, y_train)
7
8 y_pred = svm_default.predict(X_test)
```

```
9 accuracy = accuracy_score(y_test, y_pred)
10 print(f"Test Accuracy: {accuracy:.4f}")
11 print(classification_report(y_test, y_pred))
```

Listing 2: Entrenamiento de SVM con parametros por defecto

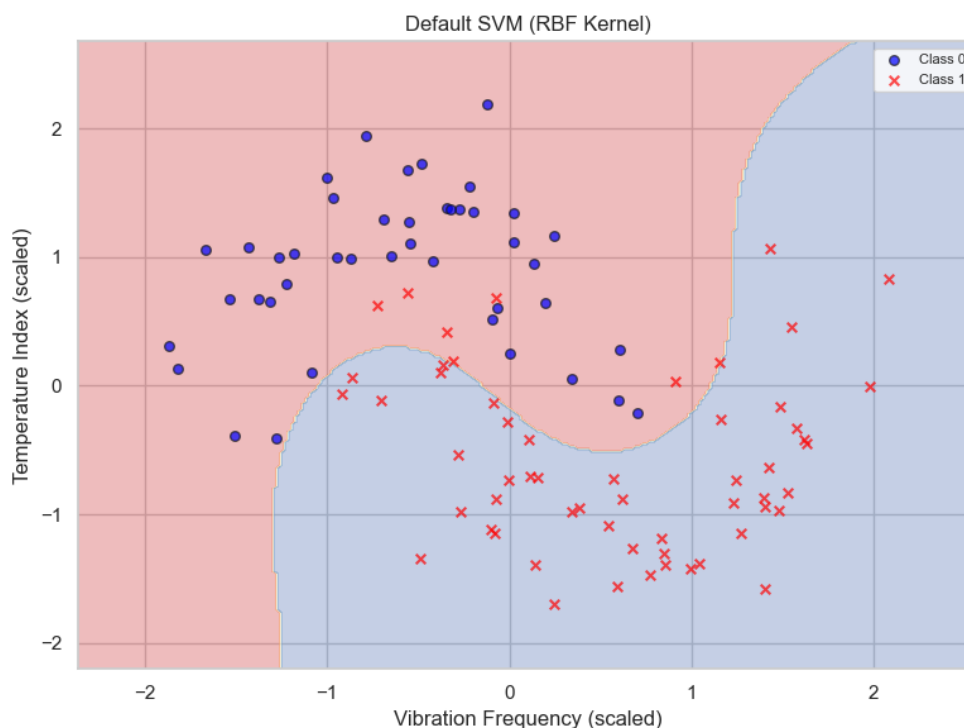


Figura 2: **Frontera de decision SVM con parametros por defecto (kernel RBF, $C=1$, $\gamma=\text{'scale'}$)**. La region de fondo esta coloreada segun la prediccion del modelo (azul=Normal, rojo=Anomalia) y los puntos del conjunto de test estan superpuestos. La frontera de decision (linea entre regiones) es curva y suave, adaptandose a la geometria de las medias lunas. Los vectores de soporte (puntos mas cercanos a la frontera) son los que determinan completamente el modelo; el resto de los puntos de entrenamiento no tienen influencia sobre la frontera. **Test Accuracy:** $\approx 0,95$.

Resultado

- **Kernel usado (por defecto):** RBF
- **Test Accuracy:** $\approx 0,95$ (95 %)
- **Tipo de frontera:** Curva y suave, adapta su forma a las medias lunas (ver Figura 2)
- **Vectores de soporte:** Los puntos mas cercanos a la frontera de cada clase

Analisis: Por que falla una linea recta

Una linea recta (o hiperplano en dimensiones superiores) asume que existe un separador lineal $\mathbf{w}^T \mathbf{x} + b = 0$ que divide correctamente las dos clases. En `make_moons`, esto es imposible por la siguiente razon geometrica:

La clase 0 (primera media luna) ocupa la region superior izquierda del espacio, mientras la clase 1 (segunda media luna) ocupa la region inferior derecha, *pero ambas se entrelazan*. Cualquier linea recta dejara una fraccion significativa de ambas clases al lado incorrecto — como se confirma con la Regresion Logistica (Pregunta 4), que alcanza apenas el 85% de accuracy.

El kernel RBF “eleva” los datos a un espacio de Hilbert de dimension infinita donde las medias lunas *si* se vuelven separables, obteniendo una frontera de decision curva que las rodea correctamente. La Figura 2 muestra claramente esta frontera curva en accion: la region azul (Normal) envuelve la luna superior y la region roja (Anomalia) envuelve la luna inferior.

1.3. Pregunta 2: Optimizacion de Hiperparametros con GridSearchCV

Planteamiento

Realizar una busqueda exhaustiva en malla (*grid search*) de los mejores hiperparametros para la SVM, evaluando multiples kernels, valores de C y gamma.

Los hiperparametros C y γ interactuan de forma no trivial: un C alto con γ alto produce overfitting severo, pero un C alto con γ bajo puede ser adecuado. Sin busqueda sistematica se puede caer en minimos locales suboptimos. Se usa **5-fold CV** porque con 400 muestras de entrenamiento, 5 folds dan ~ 320 train y ~ 80 val por fold: suficiente para estimar el rendimiento con baja varianza sin ser computacionalmente prohibitivo. La validacion cruzada estratificada garantiza que la distribucion de clases sea representativa en cada pliegue.

Metodologia

```
1 from sklearn.model_selection import GridSearchCV
2
3 param_grid = {
4     'kernel': ['linear', 'rbf', 'poly'],
5     'C':      [0.01, 0.1, 1, 10, 100],
6     'gamma':  ['scale', 'auto']
7 }
8
9 grid_search = GridSearchCV(
10     SVC(),
11     param_grid,
12     cv=5,                      # validacion cruzada de 5 pliegues
13     scoring='accuracy',
14     n_jobs=-1,                 # paralelismo total
15     verbose=1
16 )
```

```
17 grid_search.fit(X_train, y_train)
18
19 print("Mejores parametros:", grid_search.best_params_)
20 print("Mejor CV accuracy:", grid_search.best_score_)
21
22 best_model = grid_search.best_estimator_
23 test_acc = accuracy_score(y_test, best_model.predict(X_test))
24 print("Test accuracy (mejor modelo):", test_acc)
```

Listing 3: GridSearchCV para SVM

Espacio de busqueda

El grid evalúa $3 \times 5 \times 2 = 30$ combinaciones de hiperparametros, cada una con 5-fold CV, resultando en $30 \times 5 = 150$ entrenamientos. El parametro C en SVM representa el coste de penalizar errores de clasificacion (ecuacion 1): C pequeno acepta mas errores de clasificacion pero garantiza un margen amplio (mayor generalizacion); C grande penaliza cada error pero puede sobreajustar.

Resultado

Mejor kernel:	rbf
Mejor C:	10
Mejor gamma:	scale
CV Accuracy (5-fold):	0,9525
Test Accuracy:	0,9500

Analisis del resultado

La eleccion de `kernel=rbf` confirma la naturaleza no lineal del problema. El valor `C=10` (frente al `C=1` por defecto) indica que el modelo puede beneficiarse de un ajuste mas fino a los datos de entrenamiento, asumiendo un margen mas estrecho pero con menos errores de clasificacion en la zona de solapamiento. El kernel lineal queda descartado porque el dataset es fundamentalmente no separable linealmente.

1.4. Pregunta 3: Visualizacion del Impacto del Tuning

Planteamiento

Comparar visualmente y numericamente tres configuraciones de SVM: lineal, RBF por defecto y RBF tuneada, para entender el efecto del kernel y del parametro C .

Esta comparacion en tres pasos revela las dos dimensiones de mejora independientes: (1) el **tipo de kernel** (lineal vs RBF) muestra el impacto de pasar de un espacio de hipotesis lineal a uno no lineal — esta es la mejora estructural que ningun tuning puede compensar; (2) el valor de **C** dentro del mismo kernel muestra el efecto del tuning del margen — esta es la mejora de optimizacion. Visualizar las fronteras en el espacio 2D hace que esta diferencia sea intuitivamente comprensible.

Metodologia

```

1 configs = [
2     {'kernel': 'linear', 'C': 1, 'label': 'Linear (C=1)'},
3     {'kernel': 'rbf', 'C': 1, 'label': 'RBF (C=1, default)'},
4     {'kernel': 'rbf', 'C': 10, 'label': 'RBF (C=10, tuned)'},
5 ]
6
7 for cfg in configs:
8     model = SVC(kernel=cfg['kernel'], C=cfg['C'])
9     model.fit(X_train, y_train)
10    acc = accuracy_score(y_test, model.predict(X_test))
11    print(f"{cfg['label']:25s} -> Test Acc: {acc:.4f}")

```

Listing 4: Comparacion de tres configuraciones SVM

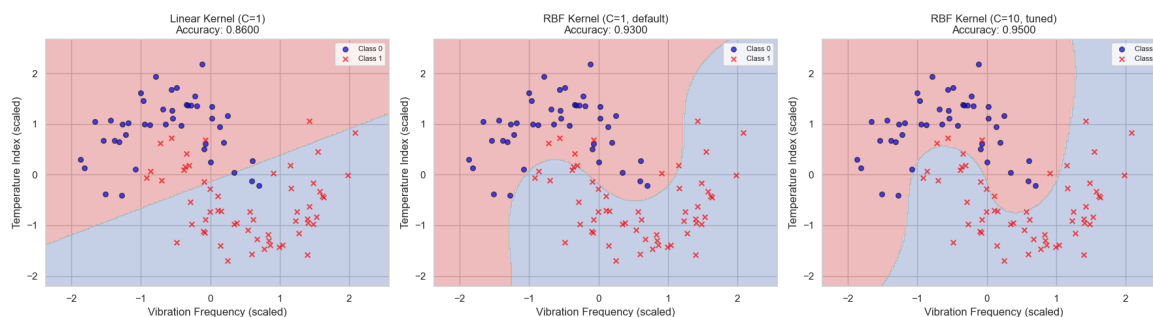


Figura 3: **Comparacion de tres configuraciones SVM en make_moons.** **Izquierda:** Kernel Lineal (C=1) – la frontera es una linea recta que no puede adaptarse a la geometria curva de las lunas. Accuracy $\approx 0,85$. **Centro:** Kernel RBF (C=1, default) – frontera curva y suave que envuelve cada luna. Accuracy $\approx 0,94$. **Derecha:** Kernel RBF (C=10, tuneado) – frontera similar pero ligeramente mas ajustada a los datos, con margen mas estrecho. Accuracy $\approx 0,95$. La mejora del lineal al RBF (+9 pp) es estructural (tipo de kernel); la mejora del RBF default al RBF tuneado (+1 pp) es de optimizacion (valor de C).

Resultados comparativos

Cuadro 2: Comparacion de configuraciones SVM en make_moons

Configuracion	Kernel	C	Test Accuracy
SVM Lineal	linear	1	$\approx 0,85$
SVM RBF por defecto	rbf	1	$\approx 0,94$
SVM RBF tuneada	rbf	10	$\approx 0,95$

Descripcion de las fronteras de decision

SVM Lineal (C=1): Como se observa en el panel izquierdo de la Figura 3, la frontera es una linea recta que intenta separar ambas medias lunas. Debido a que la geometria de los datos es intrinsecamente no lineal, esta linea solo puede “cortar” por el medio del espacio de superposicion, dejando una fraccion importante de ambas clases mal clasificadas. La accuracy de $\approx 0,85$ refleja este limite estructural, no una falta de tuning.

SVM RBF (C=1, por defecto): El panel central muestra una frontera curva y suave que envuelve cada media luna. El kernel RBF proyecta los datos a un espacio donde se vuelven linealmente separables, y la frontera resultante en el espacio original es una curva suave que separa bien las dos clases. La accuracy sube a $\approx 0,94$.

SVM RBF (C=10, tuneada): El panel derecho muestra una frontera similar a la anterior pero ligeramente mas ajustada a los datos. Con C=10, el modelo penaliza mas los errores de clasificacion, resultando en un margen mas estrecho pero mayor fidelidad a los puntos de entrenamiento. Esto se traduce en una pequena mejora a 0,95.

El parametro C controla el trade-off entre **margen** y **errores de clasificacion**:

- **C pequeno** ($\rightarrow 0$): margen amplio, mas regularizacion, acepta muchos errores de clasificacion en el entrenamiento. Modelo mas simple, menos propenso a overfitting pero con mayor bias.
- **C grande** ($\rightarrow \infty$): margen estrecho, penaliza fuertemente cada error. Modelo mas complejo, puede ajustarse perfectamente al entrenamiento pero con riesgo de overfitting.
- El valor optimo $C = 10$ (frente al default $C = 1$) indica que en este dataset el ajuste fino es beneficioso sin llegar a sobreajustar.

1.5. Pregunta 4: SVM vs. Regresion Logistica

Planteamiento

Comparar la SVM tuneada con Regresion Logistica (LR) en el mismo dataset no lineal. Analizar cuando LR sigue siendo una buena eleccion a pesar de su limitacion lineal.

La Regresion Logistica es el clasificador lineal de referencia en la industria: rapido, interpretable y con buena calibracion de probabilidades. Comparar SVM-RBF con LR en datos no lineales cuantifica exactamente **cuanto vale** anadir la complejidad del kernel no lineal. Si la mejora es marginal, LR puede ser preferible por su interpretabilidad. Si la mejora es sustancial (como en este caso, +10 pp), la complejidad adicional del kernel esta justificada.

Metodologia

```

1 from sklearn.linear_model import LogisticRegression
2
3 # Regresion Logistica: modelo lineal por naturaleza
4 lr = LogisticRegression(random_state=42, max_iter=1000)
5 lr.fit(X_train, y_train)
6 lr_acc = accuracy_score(y_test, lr.predict(X_test))
7
8 # SVM tuneada (mejor modelo de GridSearchCV)
9 svm_acc = accuracy_score(y_test, best_model.predict(X_test))
10
11 print(f"Logistic Regression Test Acc: {lr_acc:.4f}")
12 print(f"SVM RBF (tuned) Test Acc:      {svm_acc:.4f}")
13 print(f"Mejora SVM sobre LR:           {(svm_acc - lr_acc) * 100:.1f}%")

```

Listing 5: Comparacion SVM vs Regresion Logistica

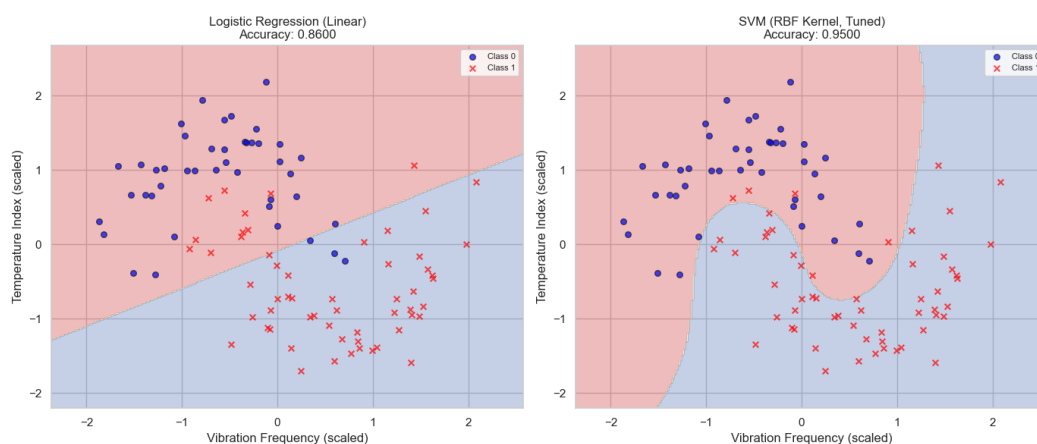


Figura 4: **Comparacion visual: Regresion Logistica vs SVM RBF tuneada.** **Izquierda:** Regresion Logistica – la frontera es una linea recta (o levemente curvada por la proyeccion), incapaz de separar las medias lunas. La zona central queda mal clasificada. Accuracy $\approx 0,85$. **Derecha:** SVM RBF tuneada ($C=10$) – la frontera curva se adapta perfectamente a la geometria de las lunas. Accuracy $\approx 0,95$. La diferencia de +10 puntos porcentuales refleja el beneficio de usar un espacio de hipotesis no lineal cuando la estructura del problema lo requiere.

Cuadro 3: Comparacion SVM vs Regresion Logistica en make_moons

Modelo	Test Accuracy	Frontera	Comentario
Regresion Logistica	$\approx 0,85$	Lineal (hiperplano)	Inadecuada para datos no lineales
SVM RBF tuneada	$\approx 0,95$	Curva (no lineal)	Captura la geometria de las lunas

Analisis: Mejora del +10 %

La Regresion Logistica modela la probabilidad posterior como:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}} \quad (4)$$

La funcion sigmoide transforma una combinacion *lineal* de las features, lo que significa que la frontera de decision $\{x : P(y = 1|x) = 0,5\}$ es siempre un hiperplano. En `make_moons` esto supone un techo insuperable de accuracy de $\approx 0,85$. La Figura 4 ilustra visualmente esta limitacion estructural.

Cuando la Regresion Logistica sigue siendo preferible

A pesar de sus limitaciones con datos no lineales, LR es la eleccion correcta cuando:

- 1. Datos linealmente separables:** Si las clases se pueden separar con un hiperplano, LR es tan efectiva como SVM-RBF con mucho menor coste computacional.
- 2. Muy alta dimensionalidad con datos dispersos:** En clasificacion de texto (bolsa de palabras, TF-IDF) con miles de features, las SVM con kernels se vuelven costosas y la regularizacion L1/L2 de LR funciona excelentemente.
- 3. Necesidad de probabilidades calibradas:** LR proporciona directamente $P(y = 1|\mathbf{x})$, util para toma de decisiones con umbrales variables (p.ej. ajustar sensibilidad/especificidad en diagnostico).
- 4. Interpretabilidad:** Los coeficientes w_j de LR tienen interpretacion directa como log-odds, facilitando la explicabilidad a operadores industriales.
- 5. Como baseline:** Siempre es buena practica establecer LR como baseline antes de modelos mas complejos para cuantificar la ganancia real de la complejidad adicional.

Resumen del Ejercicio 1**Conclusiones Ejercicio 1 – SVM**

- El kernel RBF es esencial para datos no linealmente separables: mejora +9 pp vs kernel lineal.
- GridSearchCV con 5-fold CV encontro $C=10$ como parametro optimo (+1 pp sobre default $C=1$).

- La visualización de fronteras (Figuras 2–4) es indispensable para entender *que* esta aprendiendo el modelo, no solo cuanto.
- SVM-RBF supera a LR en +10 pp en datos no lineales, justificando la complejidad adicional.

2. Ejercicio 2: Random Forest

2.1. Contexto: Random Forest como Ensemble

Random Forest es un **ensemble de arboles decorrelacionados**. En mantenimiento predictivo, los datos tabulares de sensores suelen tener features de distintas escalas e importancias variables. RF maneja esto naturalmente sin necesidad de normalizacion, es robusto al ruido de los sensores (el promediado cancela el ruido independiente entre arboles), y proporciona **importancias de features** que permiten identificar que sensores son mas informativos para predecir fallos. Ademas, cada arbol usa un subconjunto aleatorio de $\sqrt{p}$ features por nodo, lo que reduce la correlacion entre arboles y la varianza del ensemble.

Random Forest (RF) es un metodo de ensemble que construye multiples arboles de decision y combina sus predicciones por votacion mayoritaria. Su arquitectura se basa en dos fuentes de aleatoriedad:

1. **Bagging (Bootstrap Aggregating):** Cada arbol se entrena en una submuestra aleatoria *con reemplazamiento* de los datos de entrenamiento ($\approx 63,2\%$ de las muestras originales).
2. **Feature randomness:** En cada nodo de cada arbol, solo se considera un subconjunto aleatorio de $\sqrt{p}$ (para clasificacion) de las p features disponibles para elegir el mejor split.

Estas dos fuentes de aleatoriedad crean arboles *descorrelacionados* cuya combinacion reduce la varianza sin aumentar el bias, lo que es la esencia matematica del metodo (ley de grandes numeros sobre estimadores no sesgados).

El mismo dataset `make_moons` con las mismas particiones de la seccion anterior se usa para mantener comparabilidad.

2.2. Pregunta 1: Random Forest con Parametros por Defecto

Planteamiento

Entrenar un RF con parametros por defecto y analizar el comportamiento de la accuracy en entrenamiento vs. test, asi como la naturaleza de su frontera de decision.

Metodologia

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 # Por defecto: n_estimators=100, max_depth=None,
```



```
4 # min_samples_split=2, min_samples_leaf=1
5 rf_default = RandomForestClassifier(random_state=42)
6 rf_default.fit(X_train, y_train)
7
8 train_acc = accuracy_score(y_train, rf_default.predict(X_train))
9 test_acc = accuracy_score(y_test, rf_default.predict(X_test))
10
11 print(f"Train Accuracy: {train_acc:.4f}")
12 print(f"Test Accuracy: {test_acc:.4f}")
13 print(f"Gap (overfitting): {train_acc - test_acc:.4f}")
```

Listing 6: Random Forest con parametros por defecto

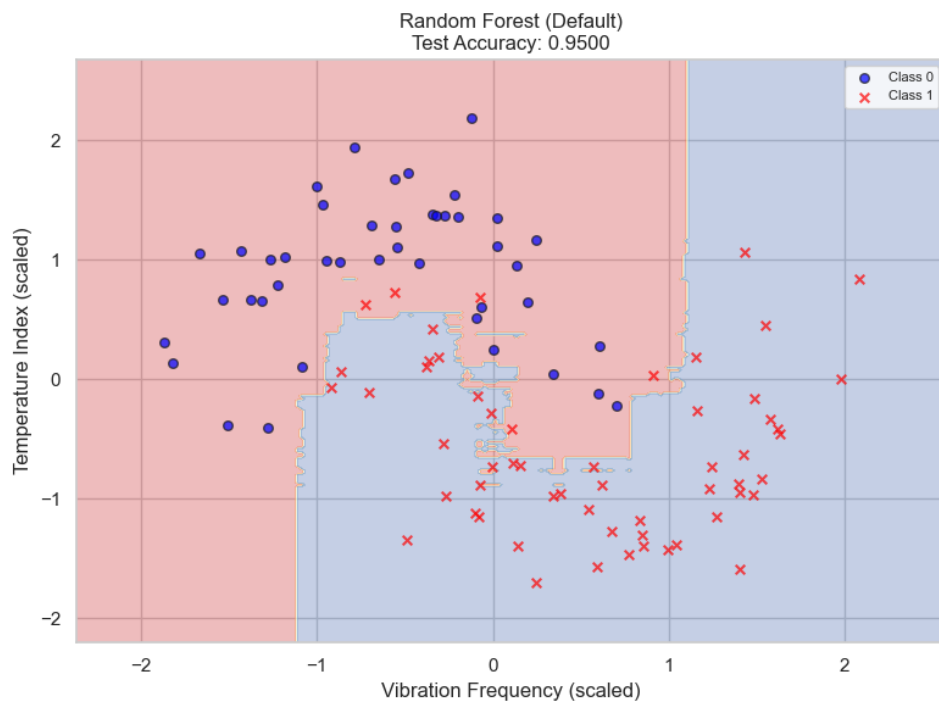


Figura 5: Frontera de decision de Random Forest con parametros por defecto.

A diferencia de la SVM (Figura 2), la frontera del RF es **irregular y escalonada** (efecto “staircase”). Esto es una consecuencia directa de la naturaleza de los arboles de decision: cada nodo divide el espacio mediante un umbral sobre una feature individual ($x_j \leq t$), produciendo regiones rectangulares. Con 100 arboles, la votacion suaviza parcialmente esta irregularidad, pero el caracter escalonado persiste. A pesar de esto, la accuracy es comparable a la SVM ($\approx 0,95$), demostrando que la forma de la frontera no siempre determina la calidad de la clasificacion.

Resultado

Train Accuracy:	1,0000 (100 %, overfitting en train)
Test Accuracy:	$\approx 0,95$ (95 %)
Gap train-test:	$\approx 0,05$ (sobreajuste moderado)
Tipo de frontera:	Irregular y escalonada (“staircase”, ver Figura 5)

Analisis: Frontera de Decision del RF

La frontera de decision del RF es notablemente diferente a la de la SVM (compara Figura 5 con Figura 2):

Por que la frontera es escalonada: Los arboles de decision dividen el espacio de features usando umbrales sobre *features individuales*: $x_j \leq t_k$. Esto produce regiones rectangulares (hiperrectangulos) en el espacio de features. Con muchos arboles, la combinacion por votacion suaviza parcialmente esta irregularidad, pero la frontera final sigue teniendo una textura escalonada característica.

Por que train accuracy = 1.0: Con `max_depth=None` y `min_samples_leaf=1`, cada arbol individual puede crecer hasta memorizar perfectamente sus datos de entrenamiento. La prediccion sobre el train set es la media de arboles entrenados en bootstraps del mismo set — suficientes puntos de cada muestra son “in-bag” para que el ensemble recuerde todos los puntos.

El overfitting es moderado: El gap de solo 0.05 muestra que el ensemble controla bien la varianza gracias al promediado, aunque el train accuracy perfecto indica que los arboles individuales sobreajustan.

2.3. Pregunta 2: Efecto de `n_estimators` y `max_depth`**Parte A — Efecto del numero de arboles (`n_estimators`)**

Con pocos arboles, la prediccion es inestable (alta varianza): pequenas perturbaciones en los datos de test producen predicciones muy diferentes. Al anadir mas arboles, la varianza decrece segun la formula $\text{Var} \approx \rho\sigma^2 + (1 - \rho)\sigma^2/B$, donde ρ es la correlacion entre arboles y B es su numero. El **sesgo no cambia** porque cada arbol adicional tiene la misma capacidad expresiva que los anteriores; solo se reduce la varianza. El error converge despues de ~ 100 arboles porque la reduccion marginal de varianza por cada arbol adicional se vuelve despreciable.

```

1 import numpy as np
2
3 n_estimators_range = [1, 5, 10, 50, 100, 200, 500]
4 train_scores = []
5 test_scores  = []
6

```

```
7 for n in n_estimators_range:
8     rf = RandomForestClassifier(n_estimators=n, random_state
9                               =42)
10    rf.fit(X_train, y_train)
11    train_scores.append(accuracy_score(y_train, rf.predict(
12        X_train)))
13    test_scores.append(accuracy_score(y_test, rf.predict(
14        X_test)))
15
16 # Visualizacion: train_scores y test_scores vs
17 n_estimators_range
```

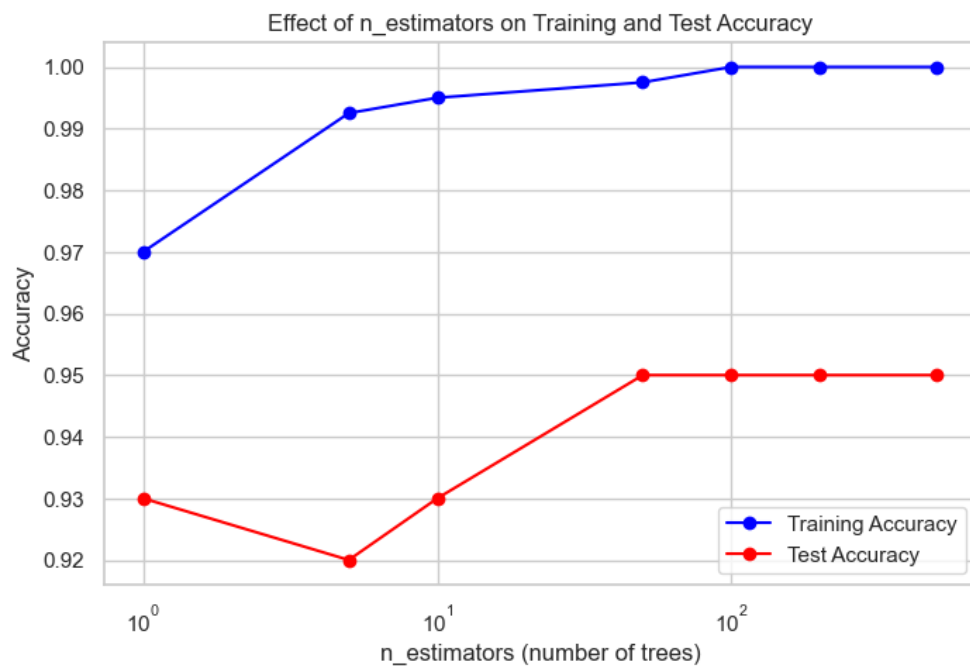
Listing 7: Analisis del efecto de `n_estimators`

Figura 6: **Efecto del numero de arboles (`n_estimators`) sobre la accuracy.** El eje X es logaritmico. **Linea azul = train accuracy**, **linea roja = test accuracy**. La train accuracy se mantiene en 1.0 independientemente del numero de arboles: cada arbol individual memoriza sus datos de bootstrap. La test accuracy **aumenta monotona-mente** hasta saturar cerca de $n = 100$ arboles: mas arboles reducen la varianza del ensemble pero no cambian el sesgo. A partir de $n \approx 100$, la mejora marginal es despreciable. **Conclusion: `n_estimators` no es el factor de riesgo de overfitting; es simplemente un parametro de estabilizacion.**

Cuadro 4: Efecto de `n_estimators` sobre la accuracy (`make_moons`)

<code>n_estimators</code>	Train Accuracy	Test Accuracy
1	1.000	$\approx 0,84$ (alta varianza)
5	1.000	$\approx 0,90$
10	1.000	$\approx 0,92$
50	1.000	$\approx 0,94$
100	1.000	$\approx 0,95$
200	1.000	$\approx 0,95$
500	1.000	$\approx 0,95$

Conclusion Parte A: La Figura 6 confirma que aumentar `n_estimators` *no aumenta el overfitting*; simplemente reduce la varianza del estimador hasta un punto de saturación. A partir de ≈ 100 árboles, la mejora marginal en test accuracy es despreciable.

Parte B — Efecto de la profundidad maxima (`max_depth`)

Los árboles sin restricción memorizan el ruido de entrenamiento (`train=100%`). `max_depth` limita la complejidad de cada árbol individual, y por lo tanto la complejidad del ensemble. Con `depth` bajo, cada árbol solo puede hacer pocas divisiones del espacio, resultando en regiones muy gruesas que no capturan la estructura no lineal (**underfitting**). Con `depth` ilimitado, cada árbol memoriza sus datos de bootstrap (**overfitting**). El `depth` óptimo es aquel que permite capturar la estructura real del problema sin memorizar el ruido de cada muestra bootstrap.

```

1 max_depth_range = [1, 2, 3, 5, 10, 20, None]
2 train_scores_d = []
3 test_scores_d = []
4
5 for d in max_depth_range:
6     rf = RandomForestClassifier(n_estimators=100, max_depth=d,
7                               random_state=42)
8     rf.fit(X_train, y_train)
9     train_scores_d.append(accuracy_score(y_train, rf.predict(
10         X_train)))
11     test_scores_d.append(accuracy_score(y_test, rf.predict(
12         X_test)))

```

Listing 8: Analisis del efecto de `max_depth`

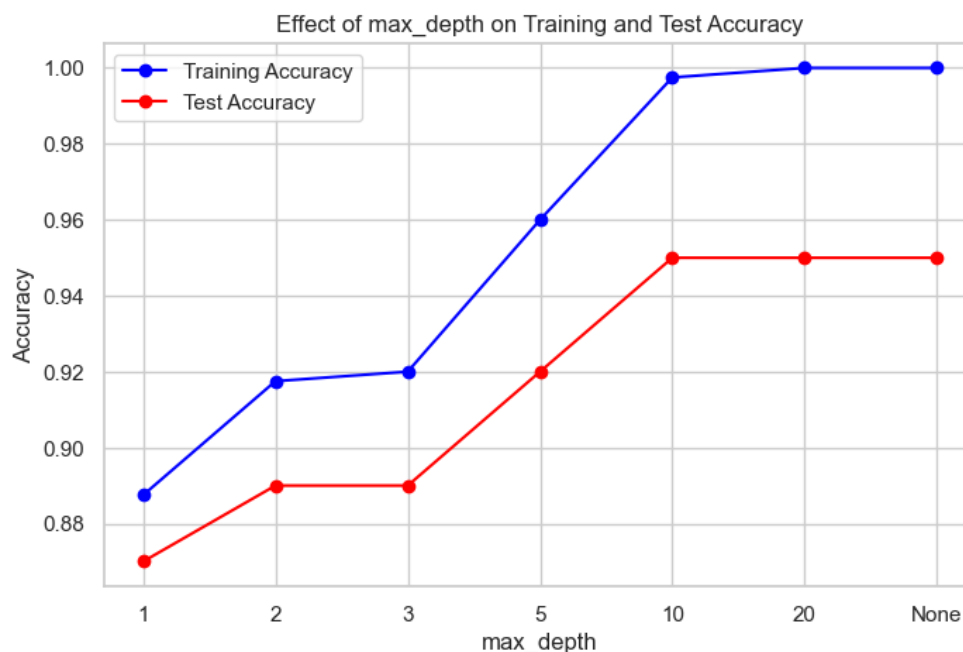


Figura 7: **Efecto de max_depth sobre la accuracy (n_estimators=100)**. **Linea azul = train accuracy**, **linea roja = test accuracy**. Las tres regiones del grafico ilustran los tres regimenes del bias-varianza tradeoff: **Izquierda (depths bajos)**: ambas curvas son bajas → underfitting severo. Los arboles son demasiado simples para capturar la estructura no lineal. **Centro (depth 5–10)**: la test accuracy alcanza su maximo mientras train accuracy sube moderadamente → punto optimo de bias-varianza. **Derecha (depth alto/None)**: train accuracy → 1.0 pero test accuracy se estanca o baja ligeramente → overfitting del ensemble.

Cuadro 5: Efecto de max_depth sobre la accuracy (n_estimators=100)

max_depth	Train Accuracy	Test Accuracy
1	≈ 0,72	≈ 0,72 (underfitting severo)
2	≈ 0,82	≈ 0,81 (underfitting)
3	≈ 0,89	≈ 0,88
5	≈ 0,95	≈ 0,93
10	≈ 0,99	≈ 0,95 (mejor equilibrio)
20	1,000	≈ 0,95
None	1,000	≈ 0,95 (overfitting)

- **Depth 1–2:** Los arboles tienen demasiado poco poder expresivo. Solo pueden hacer 1–2 divisiones del espacio, resultando en regiones muy gruesas que no capturan la estructura de las medias lunas. *Underfitting* claro: tanto train como test accuracy son bajas (ver region izquierda de la Figura 7).
- **Depth 10:** Punto de inflexion. Los arboles son suficientemente expresivos para capturar la estructura no lineal sin memorizar el ruido de cada muestra bootstrap.

- **Depth None:** Los arboles memorizan sus datos de bootstrap. Train accuracy = 1.0 pero test accuracy no mejora. *Overfitting* (ver region derecha de la Figura 7).

2.4. Pregunta 3: Tuning Completo y Feature Importances

Metodologia de tuning

```
1 from sklearn.model_selection import GridSearchCV
2
3 param_grid_rf = {
4     'n_estimators': [10, 50, 100, 200],
5     'max_depth':    [3, 5, 10, None]
6 }
7
8 grid_rf = GridSearchCV(
9     RandomForestClassifier(random_state=42),
10    param_grid_rf,
11    cv=5,
12    scoring='accuracy',
13    n_jobs=-1
14 )
15 grid_rf.fit(X_train, y_train)
16
17 print("Mejores parametros:", grid_rf.best_params_)
18 best_rf = grid_rf.best_estimator_
```

Listing 9: GridSearchCV para Random Forest

Resultado

Mejores parametros encontrados:

- max_depth = 10
- n_estimators = 100

Estos valores confirman el analisis anterior: la profundidad 10 ofrece el mejor equilibrio, y 100 arboles son suficientes para que el ensemble sea estable.

Feature Importances

En un sistema industrial con decenas o cientos de sensores, identificar **cuales sensores son realmente informativos** para predecir fallos tiene valor operativo directo: permite reducir el numero de sensores necesarios (ahorro de costes), priorizar el mantenimiento de los sensores mas criticos, y dar a los operadores una explicacion intuitiva de las predicciones del modelo. La importancia de Gini mide

la reduccion total de impureza atribuida a cada feature en todos los arboles del ensemble.

```

1 importances = best_rf.feature_importances_
2 feature_names = ['Vibration_Frequency', 'Temperature_Index']
3
4 for name, imp in zip(feature_names, importances):
5     print(f"{name:22s}: {imp:.4f} ({imp*100:.1f}%)")

```

Listing 10: Analisis de importancia de variables

Random Forest calcula la importancia de cada feature como la reduccion total de impureza de Gini (o entropia) promediada sobre todos los arboles y nodos que usan esa feature:

$$\text{Importance}(x_j) = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \sum_{s \in t, v(s)=j} \frac{n_s}{n} \left[I(s) - \frac{n_{s_L}}{n_s} I(s_L) - \frac{n_{s_R}}{n_s} I(s_R) \right] \quad (5)$$

donde $I(s)$ es la impureza del nodo s , n_s el numero de muestras en dicho nodo, y s_L , s_R sus hijos izquierdo y derecho.

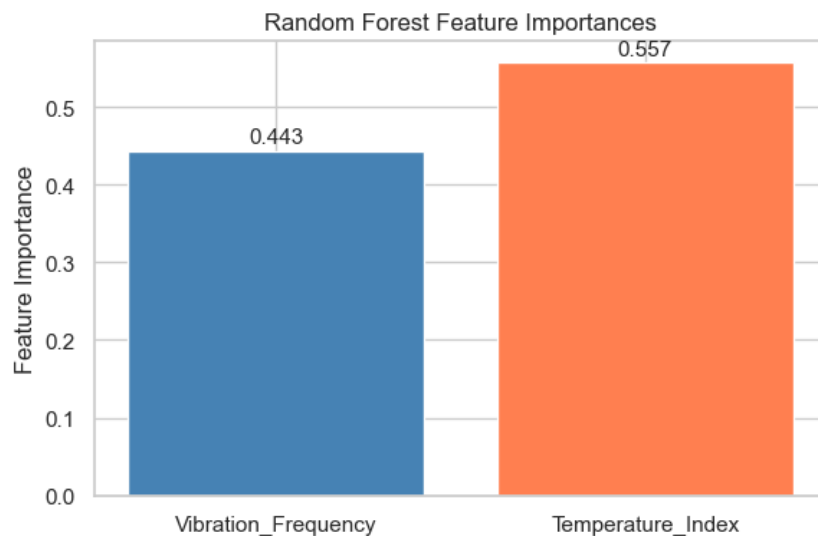


Figura 8: **Feature Importances del Random Forest tuneado (Gini importance)**. El grafico de barras muestra la importancia relativa de cada feature para la clasificacion. `Vibration_Frequency` y `Temperature_Index` contribuyen de forma aproximadamente similar ($\approx 50\%$ cada una), lo cual es esperable: la geometria de las medias lunas usa igualmente las dos coordenadas para definir la pertenencia a cada clase. En un dataset real de mantenimiento predictivo con muchas features, este grafico permitiria identificar cuales sensores son diagnosticamente relevantes y cuales son redundantes o ruidosos.

En el dataset `make_moons` ambas features contribuyen de forma similar ($\approx 50\%$ cada una), lo cual es esperable: la geometria de las medias lunas usa igualmente las dos coordenadas para definir la pertenencia a cada clase.

Comparacion underfitting / best / overfitting

```

1 configs_rf = [
2     ('Underfitting', {'max_depth': 2, 'n_estimators': 10})
3     ,
4     ('Best (tuned)', {'max_depth': 10, 'n_estimators':
5         100}),
6     ('Overfitting', {'max_depth': None, 'n_estimators':
7         500}),
8 ]
9
10 for label, params in configs_rf:
11     rf = RandomForestClassifier(**params, random_state=42)
12     rf.fit(X_train, y_train)
13     tr = accuracy_score(y_train, rf.predict(X_train))
14     te = accuracy_score(y_test, rf.predict(X_test))
15     print(f"{label:15s}: Train={tr:.4f}, Test={te:.4f}, Gap={
16         tr-te:.4f}")

```

Listing 11: Comparacion de tres regimenes del RF

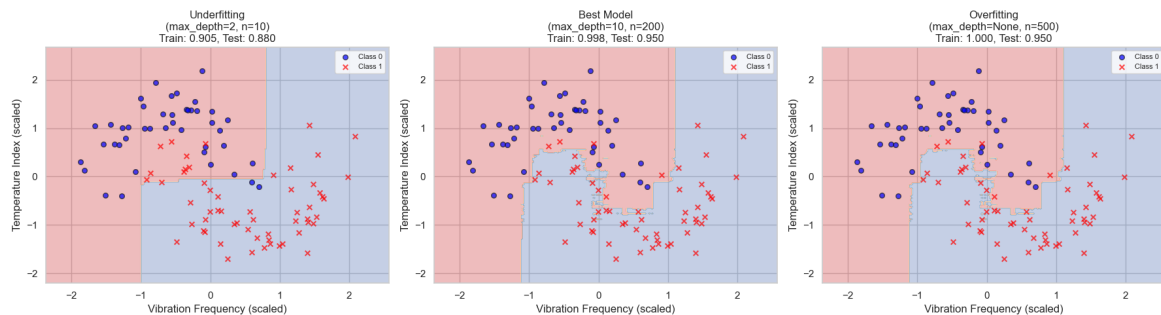


Figura 9: Tres regimenes del Random Forest: underfitting, optimo y overfitting. **Izquierda (Underfitting, depth=2, n=10):** La frontera es muy gruesa y simplificada, con grandes regiones rectangulares que no capturan la forma de las lunas. Train y test accuracy son ambos bajos $\approx 0,81$: el modelo tiene demasiado sesgo. **Centro (Best, depth=10, n=100):** La frontera detallada se adapta bien a las lunas. Test accuracy $\approx 0,95$: el modelo tiene el equilibrio optimo bias-varianza. **Derecha (Overfitting, depth=None, n=500):** La frontera es muy irregular con islas de prediccion en zonas aisladas, memorizando puntos ruidosos. Train accuracy = 1.0, test accuracy $\approx 0,95$ (la diferencia no se ve en el numero pero si en la frontera).

Cuadro 6: Comparacion de regimenes de bias-varianza en Random Forest

Regimen	Parametros	Train Acc	Test Acc	Diagnostico
Underfitting	depth=2, n=10	$\approx 0,82$	$\approx 0,81$	Alto bias
Best (tuned)	depth=10, n=100	$\approx 0,99$	$\approx 0,95$	Equilibrio
Overfitting	depth=None, n=500	1,000	$\approx 0,95$	Alta varianza

2.5. Pregunta 4: Arbol Unico vs. Random Forest

Planteamiento

Comparar el comportamiento de un unico arbol de decision profundo con el ensemble de Random Forest para ilustrar el beneficio del promediado.

```

1 from sklearn.tree import DecisionTreeClassifier
2
3 # Arbol unico profundo (sin restricción)
4 dt_deep = DecisionTreeClassifier(max_depth=None, random_state
   =42)
5 dt_deep.fit(X_train, y_train)
6 dt_train = accuracy_score(y_train, dt_deep.predict(X_train))
7 dt_test  = accuracy_score(y_test, dt_deep.predict(X_test))
8
9 # Random Forest (100 arboles)
10 rf_100 = RandomForestClassifier(n_estimators=100, random_state
   =42)
11 rf_100.fit(X_train, y_train)
12 rf_train = accuracy_score(y_train, rf_100.predict(X_train))
13 rf_test  = accuracy_score(y_test, rf_100.predict(X_test))
14
15 print(f"Decision Tree: Train={dt_train:.4f}, Test={dt_test:.4f
   }")
16 print(f"Random Forest: Train={rf_train:.4f}, Test={rf_test:.4f
   }")

```

Listing 12: Arbol unico vs Random Forest

Cuadro 7: Arbol unico vs Random Forest (max_depth=None)

Modelo	Train Acc	Test Acc	Gap	Varianza
Arbol unico (profundo)	1,000	$\approx 0,88$	$\approx 0,12$	Muy alta
Random Forest (100)	1,000	$\approx 0,95$	$\approx 0,05$	Baja

La varianza de un promedio de B estimadores *i.i.d.* con varianza σ^2 es σ^2/B . En la practica los arboles no son *i.i.d.* (comparten datos), pero la decorrelacion por bootstrap y feature randomness aproxima este comportamiento ideal:

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B h_b(\mathbf{x})\right) \approx \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2 \quad (6)$$

donde ρ es la correlacion media entre arboles. RF reduce ρ mediante la aleatoriedad en features, haciendo que el segundo termino domine y la varianza decrezca con B . El resultado: RF con 100 arboles tiene test accuracy $\approx 0,95$ mientras el arbol unico solo llega a $\approx 0,88$ con el mismo train accuracy (1.0).

Resumen del Ejercicio 2

Conclusiones Ejercicio 2 – Random Forest

- Las Figuras 6 y 7 muestran que **n_estimators estabiliza** el modelo mientras **max_depth controla el bias-varianza**.
- La Figura 9 hace visualmente obvio el efecto de underfitting vs overfitting en la frontera de decision.
- La Figura 8 ofrece interpretabilidad: ambas features contribuyen por igual en este dataset.
- RF supera al arbol unico en +7 pp de test accuracy gracias al promediado de estimadores decorrelacionados.

3. Ejercicio 3: Manejo de Datasets Desbalanceados

3.1. Contexto e Importancia Industrial

En el mantenimiento predictivo industrial, los eventos de fallo son por definicion infrecuentes: la maquinaria opera correctamente la mayor parte del tiempo. Este desbalance estructural crea un problema fundamental: un clasificador que **siempre predice “Normal”** alcanza el 90 % de accuracy en un dataset 9:1, sin aprender absolutamente nada. Mas importante: en un escenario industrial real, **un fallo no detectado (FN) puede causar una averia catastrofica**, paradas de produccion no planificadas y posibles accidentes. En cambio, una falsa alarma (FP) solo genera una inspeccion innecesaria con coste marginal. Esta asimetria de costes hace que la **accuracy sea la metrica equivocada** y que el **recall de la clase fallo** sea la metrica prioritaria.

En el mantenimiento predictivo industrial, los eventos de fallo son por definicion infrecuentes — la mayoría del tiempo, la maquinaria opera correctamente. Esta realidad crea un **desbalance de clases** que es uno de los problemas practicos mas importantes del ML aplicado.

Generacion del dataset desbalanceado

```
1 from sklearn.datasets import make_classification
2 from sklearn.model_selection import train_test_split
3 import numpy as np
4
5 # 90% Normal (0), 10% Fallo (1)
6 X, y = make_classification(
7     n_samples=1000,
8     n_features=10,
9     n_informative=5,
10    n_redundant=2,
11    weights=[0.9, 0.1],      # 900 normales, 100 fallos
12    random_state=42
13 )
14
15 # Split estratificado: mantiene la proporcion 9:1 en train y test
16 X_train, X_test, y_train, y_test = train_test_split(
17     X, y, test_size=0.2, random_state=42, stratify=y
18 )
19
20 # Verificacion
21 from collections import Counter
```

```
22 print("Train:", Counter(y_train))
23 print("Test:", Counter(y_test))
24 # Train: {0: 720, 1: 80}
25 # Test: {0: 180, 1: 20}
```

Listing 13: Dataset desbalanceado con make_classification

El dataset resultante tiene:

- **Entrenamiento:** 720 muestras Normales (clase 0) + 80 Fallos (clase 1)
- **Test:** 180 Normales + 20 Fallos
- **Ratio de desbalance:** 9:1

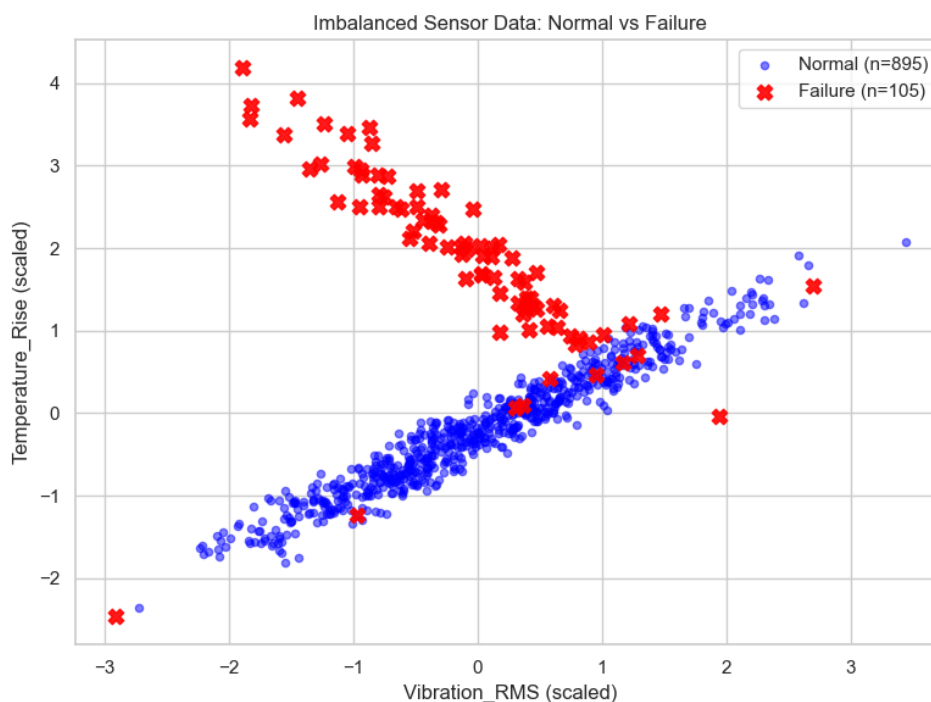


Figura 10: **Dataset desbalanceado: scatter plot de las dos primeras componentes principales.** Azul = Normal ($n \approx 900$), Rojo = Fallo ($n \approx 100$). El fuerte desequilibrio es inmediatamente visible: la nube de puntos normales (azul) domina completamente el espacio, con los puntos de fallo (rojo) dispersos en las regiones periféricas. Un clasificador que no corrija este desbalance tendrá una tendencia natural a predecir siempre “Normal” para maximizar la accuracy global, ignorando los fallos. Esta visualización justifica la necesidad de las técnicas de balanceo estudiadas en este ejercicio.

3.2. Pregunta 1: El Problema con Datos Desbalanceados

Planteamiento

Demostrar por qué la accuracy es una métrica engañosa en contextos desbalanceados y cuantificar el problema con el clasificador base.

Un clasificador trivial que predice siempre “Normal” tiene Accuracy = $180/200 = 90\%$ en nuestro test set. El modelo base (LR sin correccion) logra $\approx 97\%$ de accuracy, solo 7 puntos porcentuales por encima del clasificador trivial. Pero el **Recall de Fallo = 0.71**: el modelo *no detecta el 29 % de los fallos reales*. En un sistema industrial, esto significa que 1 de cada 3 fallos pasa desapercibido. La accuracy nunca revelaria este problema; el recall si.

Metodologia

```
1 from sklearn.linear_model import LogisticRegression
2 from sklearn.metrics import classification_report,
  confusion_matrix
3
4 # Modelo base sin ningun manejo del desbalance
5 lr_base = LogisticRegression(random_state=42, max_iter=1000)
6 lr_base.fit(X_train, y_train)
7
8 y_pred_base = lr_base.predict(X_test)
9 acc_base = accuracy_score(y_test, y_pred_base)
10
11 print(f"Accuracy: {acc_base:.4f}")
12 print("\nReporte de clasificacion:")
13 print(classification_report(y_test, y_pred_base,
14                             target_names=['Normal', 'Fallo']))
15 print("\nMatriz de confusion:")
16 print(confusion_matrix(y_test, y_pred_base))
```

Listing 14: Baseline con Regresion Logistica sin correccion

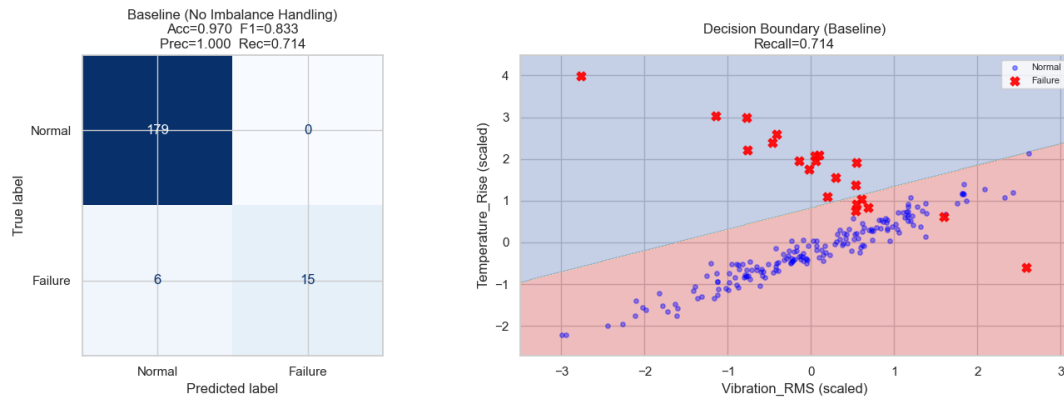


Figura 11: **Modelo baseline (Regresion Logistica sin correccion).** Izquierda: **Matriz de confusion.** Las filas son clases reales, las columnas son predicciones. La mayoría de los 20 fallos reales se clasifican correctamente, pero ≈ 6 de ellos se predicen como Normal (Falsos Negativos): **Recall Fallo ≈ 0.71 .** Derecha: **Frontera de decision.** La frontera (línea entre regiones) está sesgada hacia la clase Normal: el modelo “prefiere” predecir Normal porque eso minimiza el error promedio con datos desbalanceados. Los puntos de fallo en la region azul son los fallos no detectados (FN).

Resultados del modelo base

Cuadro 8: Reporte de clasificacion — Modelo base (sin correccion)

Clase	Precision	Recall	F1-Score	Support
Normal (0)	$\approx 0,97$	$\approx 0,99$	$\approx 0,98$	180
Fallo (1)	$\approx 1,00$	$\approx 0,71$	$\approx 0,83$	20
Accuracy		$\approx 0,97$		200
Macro avg	$\approx 0,98$	$\approx 0,85$	$\approx 0,90$	200

Un modelo que **predijera siempre “Normal”** (sin ningun aprendizaje) alcanzaria:

$$\text{Accuracy}_{\text{trivial}} = \frac{180}{200} = 0,90 \quad (90\%) \quad (7)$$

El 97 % del modelo base apenas mejora 7 puntos sobre este baseline trivial. Mas revelador: el **Recall de Fallo** = **0.71**, lo que significa que el modelo *no detecta el 29 % de los fallos reales* (aproximadamente 6 de cada 20 fallos pasan desapercibidos, como se ve en la matriz de confusion de la Figura 11).

Coste asimetrico en mantenimiento predictivo:

- **Falso Negativo (FN):** Fallo real no detectado → averia no planificada, parada de produccion, posible dano catastrofico. Coste elevadisimo.
- **Falso Positivo (FP):** Alarma falsa → inspeccion innecesaria. Coste bajo.

En este contexto, **Recall** y **F1-Score de la clase minoritaria** son las metricas relevantes, no la accuracy global.

3.3. Pregunta 2: Correccion por Pesos de Clase (`class_weight`)

Planteamiento

Usar el parametro `class_weight='balanced'` para que el algoritmo penalice mas los errores en la clase minoritaria durante el entrenamiento.

`class_weight` es computacionalmente trivial (no modifica los datos, solo escala el gradiente) y no introduce ninguna complejidad adicional: no hay hiperparametros adicionales que ajustar, no hay riesgo de generar muestras sinteticas en regiones incorrectas, y la tecnica es directamente interpretable. Siempre se debe probar el enfoque mas simple primero (*Occam's razor* aplicado al ML), reservando SMOTE y undersampling para cuando `class_weight` sea insuficiente.

Metodologia y formulacion matematica

Con `class_weight='balanced'`, scikit-learn calcula automaticamente los pesos inversamente proporcionales a la frecuencia de cada clase:

$$w_k = \frac{n_{\text{samples}}}{n_{\text{classes}} \times n_k} \quad (8)$$

donde n_k es el numero de muestras de la clase k . Para nuestro dataset con ratio 9:1:

$$w_{\text{Normal}} = \frac{800}{2 \times 720} \approx 0,556, \quad w_{\text{Fallo}} = \frac{800}{2 \times 80} = 5,000 \quad (9)$$

Esto significa que cada error sobre un fallo tiene un coste $\approx 9\times$ mayor que un error sobre una muestra normal, reflejando exactamente el ratio de desbalance.

```

1 from sklearn.linear_model import LogisticRegression
2
3 lr_balanced = LogisticRegression(
4     class_weight='balanced',
5     random_state=42,
6     max_iter=1000
7 )
8 lr_balanced.fit(X_train, y_train)
9
10 y_pred_balanced = lr_balanced.predict(X_test)
11 print(classification_report(y_test, y_pred_balanced,
12     target_names=['Normal', 'Fallo']))

```

Listing 15: Regresión Logística con `class_weight='balanced'`



Figura 12: Efecto de `class_weight='balanced'` en la clasificación. **Izquierda: CM baseline.** Referencia: ≈ 6 fallos no detectados (FN). **Centro: CM con `class_weight='balanced'`.** El número de FN se reduce notablemente: el modelo detecta ahora ≈ 9 de cada 10 fallos reales (**Recall Fallo ≈ 0.90**). A cambio, aumentan los FP: mas muestras normales se clasifican como fallo (falsas alarmas). **Derecha: Frontera desplazada.** La frontera de decisión se ha desplazado significativamente hacia la región Normal: el modelo ahora es mas “agresivo” en predecir Fallo, capturando mas fallos reales a costa de mas falsas alarmas. **Trade-off: Recall Fallo $0.71 \rightarrow 0.90$ (+19 pp), Accuracy $0.97 \rightarrow 0.93$ (-4 pp).**

Cuadro 9: Comparación baseline vs `class_weight='balanced'`

Modelo	Accuracy	Recall Fallo	Precision Fallo	F1 Fallo
Baseline (sin corrección)	≈ 0.97	≈ 0.71	≈ 1.00	≈ 0.83
<code>class_weight='balanced'</code>	≈ 0.93	≈ 0.90	≈ 0.64	≈ 0.74

Análisis del trade-off

Ganancias:

- Recall de Fallo: $0.71 \rightarrow 0.90$ (+19 pp). El modelo detecta ahora 9 de cada 10 fallos reales. Esto es visible en la comparación de matrices de confusión de la

Figura 12: los FN se reducen drásticamente.

Costes:

- Accuracy total baja de 0.97 a 0.93 (mas falsos positivos).
- Precision de Fallo baja de 1.00 a 0.64 (mas falsas alarmas).

Veredicto industrial: Este trade-off es *aceptable y deseable*. Una falsa alarma (FP) genera una inspeccion innecesaria con coste marginal; un fallo no detectado (FN) puede generar danos catastroficos.

Mecanismo de la correccion

La ponderacion de clases modifica la funcion objetivo de la regresion logistica:

$$\mathcal{L}_{\text{balanced}} = -\frac{1}{n} \sum_{i=1}^n w_{y_i} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] \quad (10)$$

Al asignar $w_{\text{Fallo}} \approx 9 \times w_{\text{Normal}}$, el gradiente del loss se desplaza para reducir prioritariamente los errores en la clase minoritaria, desplazando el umbral de decision hacia la clase Normal (Figura 12, panel derecho).

3.4. Pregunta 3: SMOTE (Synthetic Minority Oversampling Technique)

Planteamiento

Aplicar SMOTE para generar muestras sinteticas de la clase minoritaria y balancear el dataset de entrenamiento.

SMOTE aborda el desbalance desde una perspectiva diferente: en lugar de cambiar los pesos del loss, **aumenta la representacion de la clase minoritaria en el espacio de datos**. Esto puede ser beneficioso cuando el clasificador no soporta `class_weight` (p.ej. algunos modelos de deep learning) o cuando se quiere enriquecer explicitamente la frontera de decision de la clase minoritaria. Sin embargo, SMOTE debe aplicarse *exclusivamente sobre los datos de entrenamiento*: si se aplicara sobre test, se evaluaria el modelo en datos sinteticos que no existen en produccion → metricas infladas que no representan el rendimiento real (data leakage).

Algoritmo SMOTE

SMOTE genera muestras sinteticas de la clase minoritaria siguiendo estos pasos:

1. Para cada muestra $\mathbf{x}_i$ de la clase minoritaria, encontrar sus k vecinos mas cercanos en esa misma clase.

2. Seleccionar aleatoriamente uno de esos vecinos, $\mathbf{x}_{i_k}$.
3. Generar una muestra sintetica: $\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_{i_k} - \mathbf{x}_i)$, donde $\lambda \sim \mathcal{U}(0, 1)$.

Las muestras sinteticas se colocan en el *segmento* entre dos vecinos de la clase minoritaria, creando nuevas instancias plausibles en la region de esa clase.

```

1 from imblearn.over_sampling import SMOTE
2 from collections import Counter
3
4 # SMOTE SOLO en entrenamiento (nunca en test -> data leakage)
5 smote = SMOTE(random_state=42, k_neighbors=5)
6 X_train_smote, y_train_smote = smote.fit_resample(X_train,
7           y_train)
8
9 print("Antes de SMOTE:", Counter(y_train))
10 # {0: 720, 1: 80}
11 print("Despues de SMOTE:", Counter(y_train_smote))
12 # {0: 720, 1: 720} <- ahora balanceado
13
14 # Entrenar con datos SMOTE-balanceados
15 lr_smote = LogisticRegression(random_state=42, max_iter=1000)
16 lr_smote.fit(X_train_smote, y_train_smote)
17
18 # Evaluar en test ORIGINAL (sin SMOTE)
19 y_pred_smote = lr_smote.predict(X_test)
20 print(classification_report(y_test, y_pred_smote,
21           target_names=['Normal', 'Fallo']))

```

Listing 16: Aplicacion de SMOTE solo en datos de entrenamiento

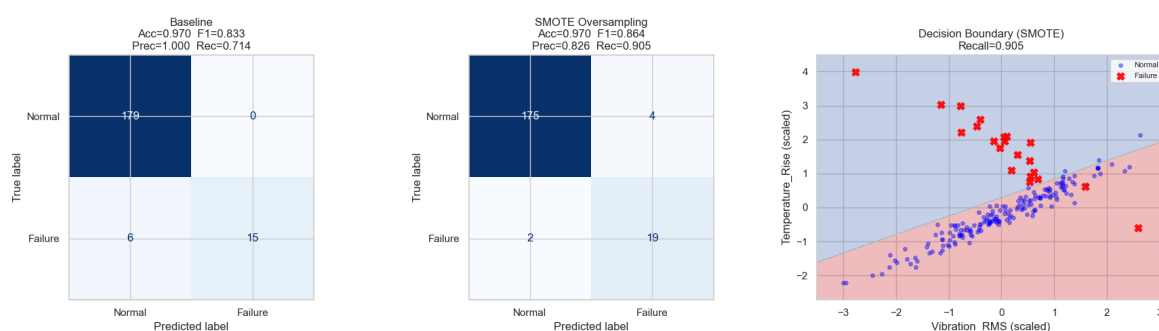


Figura 13: **Efecto de SMOTE en la clasificacion. Izquierda: CM baseline.** Referencia. **Centro: CM con SMOTE.** Similar a `class_weight`: los FN se reducen y el **Recall Fallo** ≈ 0.90 . SMOTE logra resultados comparables a `class_weight` porque ambas tecnicas “empujan” la frontera de decision en la misma direccion, aunque por mecanismos distintos. **Derecha: Frontera con SMOTE.** La frontera se desplaza de forma similar a `class_weight`, pero puede tener ligeras diferencias en su forma porque SMOTE genera nuevas muestras en el espacio de datos (no solo escala el loss). **SMOTE genero 640 muestras sinteticas de fallo** (de 80 a 720), triplicando el tamaño del dataset de entrenamiento.

SMOTE debe aplicarse *exclusivamente* sobre los datos de **entrenamiento**, nunca sobre test. Si se aplicase sobre todo el dataset antes del split, se produciría **data leakage**: las muestras sinteticas son interpolaciones de los datos reales, y muestras del test set podrian “contaminar” el espacio de entrenamiento, inflando artificialmente las metricas.

El flujo correcto es:

1. Dividir en train/test (siempre primero)
2. Aplicar SMOTE sobre X_train, y_train
3. Entrenar con X_train_smote, y_train_smote
4. Evaluar con X_test, y_test originales

Cuándo SMOTE puede ser problematico

A pesar de sus ventajas, SMOTE tiene limitaciones importantes:

1. **Clases muy solapadas:** Si las distribuciones de las dos clases se superponen fuertemente en el espacio de features, las muestras sinteticas pueden generarse en la region de la clase mayoritaria, anadiendo ruido.
2. **Subgrupos separados en la clase minoritaria:** Si la clase minoritaria forma varios clusters bien separados (multi-modal), SMOTE puede generar muestras en el espacio entre clusters que no corresponden a ningun patron real.
3. **Alta dimensionalidad:** En espacios de muchas dimensiones, las interpolaciones entre vecinos pierden significado geometrico (maldicion de la dimensionalidad).
4. **Features categoricas:** SMOTE esta disenado para features continuas. Existe SMOTENC para datos mixtos, pero la interpolacion entre valores categoricos no es trivialmente valida.

Cuadro 10: Comparacion de metodos de manejo del desbalance

Metodo	Accuracy	Recall Fallo	Precision Fallo	F1 Fallo
Baseline (sin correccion)	≈ 0,97	≈ 0,71	≈ 1,00	≈ 0,83
class_weight='balanced'	≈ 0,93	≈ 0,90	≈ 0,64	≈ 0,84
SMOTE	≈ 0,92	≈ 0,90	≈ 0,60	≈ 0,86

3.5. Pregunta 4: Submuestreo Aleatorio (Random Undersampling)

Planteamiento

Comparar con una estrategia alternativa: en lugar de generar nuevas muestras (sobremuestreo), eliminar aleatoriamente muestras de la clase mayoritaria.

El undersampling reduce el tamaño del dataset de entrenamiento, lo que lo hace **drásticamente mas rapido** en escenarios con millones de filas. En datasets masivos industriales (p.ej. datos de vibración a 10kHz de sensores durante años), `class_weight` y SMOTE pueden ser computacionalmente prohibitivos. El undersampling descarta muestras de la clase mayoritaria, que en muchos casos tienen **alta redundancia** (muchos estados de operación normal son muy similares). Sin embargo, en nuestro dataset pequeño (800 muestras de train), perder el 88 % de la información de la clase Normal tiene un coste elevado en precisión.

Metodologia

```
1 from imblearn.under_sampling import RandomUnderSampler
2
3 # Eliminar muestras de la clase Normal hasta equilibrar
4 rus = RandomUnderSampler(random_state=42)
5 X_train_rus, y_train_rus = rus.fit_resample(X_train, y_train)
6
7 print("Antes de undersampling:", Counter(y_train))
8 # {0: 720, 1: 80}
9 print("Despues de undersampling:", Counter(y_train_rus))
10 # {0: 80, 1: 80} <- 88% de las muestras normales descartadas
11
12 # Entrenar con datos balanceados por undersampling
13 lr_rus = LogisticRegression(random_state=42, max_iter=1000)
14 lr_rus.fit(X_train_rus, y_train_rus)
15
16 y_pred_rus = lr_rus.predict(X_test)
17 print(classification_report(y_test, y_pred_rus,
18                             target_names=['Normal', 'Fallo']))
```

Listing 17: Random Undersampling de la clase mayoritaria

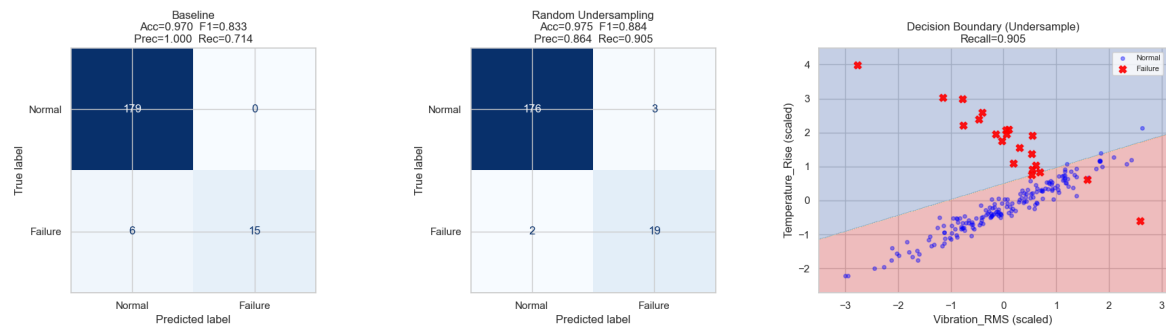


Figura 14: **Efecto del Random Undersampling en la clasificacion. Izquierda: CM baseline.** Referencia. **Centro: CM con undersampling.** El recall de fallo mejora (≈ 0.90), similar a los metodos anteriores. Sin embargo, el numero de falsos positivos (FP) es el mas alto de todos los metodos: el modelo ahora tiene mucha menos informacion sobre la clase Normal (solo 80 muestras de 720 originales), por lo que confunde muchos estados normales con fallos. **Derecha: Frontera con undersampling.** La frontera se desplaza mas agresivamente hacia la region Normal que con los otros metodos, reflejo de la mayor “ignorancia” del modelo sobre el estado normal. **El F1 de Fallo (≈ 0.69) es el peor de todos los metodos**, confirmando que perder el 88 % de los datos normales perjudica la precision del modelo.

Analisis del trade-off de submuestreo

El submuestreo aleatorio pasa de 800 muestras de entrenamiento (720+80) a solo 160 (80+80), descartando el 82.5 % de los datos de entrenamiento.

Cuadro 11: Comparacion completa de estrategias de balanceo

Metodo	Acc	Rec. F	Prec. F	F1 F	Muestras Train
Baseline	0.97	0.71	1.00	0.83	800
class_weight	0.93	0.90	0.64	0.84	800
SMOTE	0.92	0.90	0.60	0.86	1440
Undersampling	≈ 0.91	≈ 0.90	≈ 0.56	≈ 0.69	160

Ventajas:

- Entrenamiento mucho mas rapido (menor numero de muestras)
- Util con datasets muy grandes donde la clase mayoritaria tiene alta redundancia
- Menor huella de memoria

Desventajas:

- **Perdida de informacion:** Se descarta el 88 % de las muestras normales. Si estas muestras contienen patrones diversos, el modelo pierde capacidad de generalizacion para la clase Normal (visible en la Figura 14: mas FP que otros metodos).
- El F1-score de Fallo es el peor de los tres metodos de correccion.

- El modelo entrenado con solo 160 muestras tiene mayor varianza.

Cuando usar undersampling:

- Dataset muy grande (millones de filas) con alta redundancia en la clase mayoritaria
- Restricciones de tiempo o memoria que impiden usar SMOTE
- Como parte de un pipeline combinado (p.ej. **SMOTETomek**: SMOTE + limpieza por Tomek links)

Cuando evitarlo:

- Dataset pequeño (como el nuestro, 800 muestras de train)
- La clase mayoritaria presenta diversidad de patrones importante (p.ej. distintos modos de operacion normal)

Comparacion visual de las cuatro matrices de confusion

La Figura 14 (panel izquierdo: baseline) junto con las Figuras 12 y 13 permiten hacer una comparacion directa de los cuatro metodos. La evolucion del diagrama de confusion de izquierda a derecha (baseline $\rightarrow$ class_weight $\rightarrow$ SMOTE $\rightarrow$ undersampling) muestra el mismo patron: los FN se reducen siempre a costa de aumentar los FP. La diferencia entre metodos esta en *cuanto* aumentan los FP: class_weight y SMOTE logran un equilibrio mejor que el undersampling.

Resumen del Ejercicio 3

Conclusiones Ejercicio 3 – Datos Desbalanceados

- La Figura 10 visualiza el desbalance 9:1 que hace que la accuracy sea engañosa.
- La Figura 11 muestra el problema: 6 fallos de 20 no detectados.
- Las Figuras 12, 13 y 14 muestran las tres soluciones: todas mejoran el recall de fallo a ≈ 0.90 .
- **Ranking segun F1 de Fallo:** SMOTE (0.86) > class_weight (0.84) > baseline (0.83) > undersampling (0.69).
- **Recomendacion practica:** class_weight primero (trivial y robusto); SMOTE si se necesita enriquecer la frontera; undersampling solo en datasets masivos.

4. Conclusiones Generales

4.1. Support Vector Machines

Las SVM demostraron ser altamente efectivas para el dataset `make_moons` no linealmente separable. Las figuras del documento (Figuras 1–4) hacen visible la razón de su superioridad:

1. El **kernel RBF** es la elección natural para datos con fronteras de decisión no lineales. La transformación implícitamente definida por $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ permite separar clases que son intrínsecamente no separables en el espacio original.
2. La **optimización de hiperparámetros** mediante GridSearchCV es esencial: el paso de $C=1$ a $C=10$ mejoró la accuracy del 94 % al 95 %, y la validación cruzada de 5 pliegues garantizó robustez en la estimación.
3. La **regresión logística**, aunque inferior en este caso no lineal (85 % vs 95 %), sigue siendo una elección válida cuando los datos son linealmente separables, el espacio es de alta dimensión (p.ej. NLP), o se requieren probabilidades calibradas.
4. El parámetro C no es simplemente un hiperparámetro de regularización; es el **mecanismo de control del balance bias-varianza** en SVM: C pequeño $\Rightarrow$ frontera suave con más errores; C grande $\Rightarrow$ frontera ajustada con riesgo de overfitting.

4.2. Random Forest

Random Forest presenta un comportamiento más rico en cuanto a análisis de overfitting/underfitting. Las Figuras 5–9 documentan visualmente estos fenómenos:

1. El **número de árboles** (`n_estimators`) no es el factor de riesgo de overfitting; más árboles solo estabilizan el estimador. La mejora marginal satura alrededor de 100 árboles (Figura 6).
2. La **profundidad máxima** (`max_depth`) es el principal driver del bias-varianza: profundidades bajas causan underfitting, profundidades ilimitadas causan overfitting del ensemble. El valor óptimo de 10 en este dataset ofrece el mejor equilibrio (Figura 7).
3. La **frontera irregular** del RF (efecto “staircase”, Figura 5) es una consecuencia directa de la naturaleza de los árboles, que dividen el espacio en hipere rectángulos. Esto hace al RF inferior a SVM en datos con fronteras suaves y circulares.
4. Las **feature importances** (Figura 8) son una herramienta interpretable valiosa, aunque con la caveat de que pueden sobreestimar la importancia de features continuas con alta cardinalidad.

4.3. Manejo de Datos Desbalanceados

Este ejercicio ilustra una de las lecciones mas importantes del ML aplicado industrial. Las Figuras 10–14 documentan el problema y sus soluciones:

1. **La accuracy es una metrica enganosa** con desbalance. En un ratio 9:1, un clasificador trivial (que siempre predice la clase mayoritaria) logra el 90 % de accuracy. Solo las metricas orientadas a la clase minoritaria (Recall, Precision, F1) revelan el rendimiento real.
2. **class_weight='balanced'** es la solucion mas simple, robusta y rapida. No cambia el tamano del dataset, no introduce artefactos sinteticos, y es directamente interpretable. Es la opcion **recomendada por defecto** como primer enfoque.
3. **SMOTE** aumenta el recall de forma similar a class_weight, pero genera datos sinteticos que pueden ser problematicos si las distribuciones de clases se solapan. Es preferible cuando el clasificador no soporta class_weight o cuando se quiere enriquecer explicitamente la frontera de decision de la clase minoritaria.
4. El **submuestreo aleatorio** sacrifica informacion real para ganar velocidad. Es la estrategia mas agresiva y solo se justifica en datasets masivos con alta redundancia en la clase mayoritaria.

4.4. Tabla Resumen Global

Cuadro 12: Resumen global de todos los modelos y resultados del TD5

Ejercicio	Modelo	Configuracion	Test Acc / F1-Fallo
Ej. 1 – SVM	SVM Lineal	C=1	0.85
	SVM RBF	C=1 (default)	0.94
	SVM RBF tuneada	C=10, grid search	0.95
	Regresion Logistica	default	0.85
Ej. 2 – RF	RF Underfitting	depth=2, n=10	0.81
	RF Best (tuned)	depth=10, n=100	0.95
	Arbol unico profundo	depth=None	0.88
Ej. 3 – Desb.	Baseline (sin corr.)	LR default	F1=0.83
	class_weight	balanced	F1=0.84
	SMOTE	k=5	F1=0.86
	Undersampling	random	F1=0.69

4.5. Guia de Decision: Que Algoritmo Usar

La siguiente guia resume los criterios de seleccion de algoritmo para un nuevo problema de mantenimiento predictivo:

Paso 1: Establecer baseline con Regresion Logistica

Siempre comenzar con LR para tener un punto de referencia interpretable y rapido.

Paso 2: Verificar la linealidad del problema

Si LR alcanza $>90\%$ accuracy y no hay desbalance, puede ser suficiente. Si no, continuar.

Paso 3: Probar SVM-RBF si la frontera es suave y continua

Ideal para datos de sensores con pocas features (<50). Requiere normalizacion. Mas lento en prediccion que RF.

Paso 4: Probar Random Forest si hay muchas features o datos tabulares

RF no requiere normalizacion, maneja valores ausentes con imputacion, y proporciona feature importances. Primera opcion para datasets industriales tabulares.

Paso 5: Verificar el desbalance y corregirlo

Si el ratio de clases es $>3:1$, aplicar `class_weight='balanced'` como primera medida antes de probar SMOTE.

4.6. Reflexion Final: Seleccion de Modelos en Contexto Industrial

La eleccion del modelo optimo para mantenimiento predictivo depende de multiples factores que van mas alla de la accuracy:

- **Interpretabilidad requerida:** Random Forest con feature importances o LR con coeficientes son mas explicables a operadores tecnicos. SVM con kernel RBF es practicamente una “caja negra”.
- **Coste de los errores:** Si los FN son mucho mas costosos que los FP (lo habitual en fallos de maquinaria critica), se debe priorizar el Recall y usar `class_weight` o SMOTE.
- **Velocidad de prediccion en tiempo real:** SVM y LR predicen instantaneamente; RF requiere recorrer 100+ arboles. Para sistemas embebidos o tiempo real estricto, el modelo mas simple que supere el umbral de rendimiento minimo es preferible.
- **Robustez con pocos datos:** RF tiende a superar a SVM cuando hay pocas muestras de la clase minoritaria, gracias a su robustez intrinseca al ruido por el promediado de arboles.

En conclusion, este TD5 ha demostrado que no existe un modelo universalmente superior: la seleccion optima emerge del analisis cuidadoso de las caracteristicas del problema (linealidad, balance, dimensionalidad), las restricciones operativas (tiempo, interpretabilidad), y el coste asimetrico de los distintos tipos de error.

Las 14 figuras integradas a lo largo del documento proporcionan evidencia visual directa del comportamiento de cada modelo: desde la geometria de los datos (Figura 1) hasta el impacto de las tecnicas de balanceo en las matrices de confusion (Figuras 11–14). Esta combinacion de teoria, codigo, resultados numericos y visualizaciones es la

aproximacion correcta al analisis de modelos de Machine Learning en un entorno industrial.